MEDBUDDY: AI-Powered Medical Chatbot with Retrieval-Augmented Generation

Mini Project Report

Submitted by

ADITHYA BAIJU	RIE23CS006
AKHIL KUNDUVALAPPIL	RIE23CS011
DEVIKA	RIE23CS019
PAVITHRA P S	RIE23CS037
PRAKHAR SHARMA	RIE23CS039

To

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology (B.Tech) in
Computer Science and Engineering

Under the guidance of

Ms. Amrutha S Aravind



Department of Computer Science and Engineering
Rajadhani Institute of Engineering and Technology
Nagaroor, Thiruvananthapuram

March 2026

DECLARATION

I undersigned hereby declare that the mini project report **"MEDBUDDY: AI-Powered Medical Chatbot with Retrieval-Augmented Generation"**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of **Ms. Amrutha S Aravind**, Assistant Professor, Department of Computer Science & Engineering.This submission represents our ideas in our own words and where ideas or words of others have been included. We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: NAGAROOR
Date: March, 2026

Signature:
PAVITHRA P S (RIE23CS037)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJADHANI INSTITUTE OF ENGINEERING & TECHNOLOGY
NAGAROOR



CERTIFICATE

This is to certify that the report entitled '**MEDBUDDY: AI-Powered Medical Chatbot with Retrieval-Augmented Generation**' submitted by **PRAKHAR SHARMA (RIE23CS039), Adithya Baiju (RIE23CS006), Akhil Kunduvalappil (RIE23CS011), DEVIKA (RIE23CS019), and PAVITHRA P S (RIE23CS037)** to the APJ Abdul Kalam Technological University for the partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. AMRUTHE S ARAVIND
Assistant Professor Project Guide

Ms. AHTIRA SARATH
Assistant professor, project Cordinator

Dr. SANGEETHA SHIBU
Head of the Department(HoD)

ACKNOWLEDGEMENT

First and foremost, we express our heartfelt thanks and profound gratitude to the Management of Rajadhani Institute of Engineering and Technology and our Principal, **Dr. MADHUKUMAR S** for providing all the facilities to carry out this mini project work.

We are extremely happy to mention a great word of gratitude to **Dr. SANGEETHA SHIBU**, Head of Department of Computer Science and Engineering, for her valuable suggestions and encouragement to carry out this mini project work.

We would like to express our sincere gratitude to our guide **Ms. Amrutha S Aravind**, for her relevant suggestions, guidance and inestimable leadership given for the completion of this mini project work.

We thank our Mini Project Coordinator **Ms. Athira Sarath** for her constant encouragement during the entire mini project work.

Finally, we take this opportunity to express our gratitude to our parents for their love, care and support and also to our friends and well-wishers who have been constant sources of support and inspiration for completing the mini project work.

VISION OF THE INSTITUTE

Groom the Youth as Innovative, Creative and Empathetic Technologists, Hospitality Professionals, Managers and Entrepreneurs for Social Transformation.

MISSION OF THE INSTITUTE

- M1: To encourage holistic development of students with well-balanced curricular, co-curricular and extra-curricular activities.
- M2: To promote ethical and value-oriented teaching, research and consultancy among faculty and students for social transformation.
- M3: To interact with industrial organizations, governmental agencies, engineering, hospitality and business enterprises for collaborative learning.

VISION OF THE DEPARTMENT

Mould competent professionals in the field of Computer Science & Engineering.

MISSION OF THE DEPARTMENT

- M1: To develop competency in Computer Science & Engineering through outcome-based education.
- M2: To enhance research and innovations in advanced computing technology.
- M3: To inculcate social skills and entrepreneurship through collaborative learning.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

Graduates will:

- PEO 1: Exhibit knowledge and proficiency in various domains of Computer Science & Engineering.
- PEO 2: Develop innovative computing products for reconcile contemporary issues in industry.
- PEO 3: Demonstrate team work, ethical leadership and promote entrepreneurship.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

Graduates are able to:

- PSO 1: Analyse and design solutions for complex problems in computing technologies using software project management concepts and programming languages.
- PSO 2: Develop proficiency in inter-disciplinary areas of computer engineering to perform tasks related to industry & research.
- PSO 3: Solve real world problems pertaining to product and services in Information Technology.

PROGRAMME OUTCOMES (POs)

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES (Cos)

After successful completion of the course, the students will be able to:

CO1	Identify technically and economically feasible problems (Cognitive Knowledge Level: Apply).
CO2	Identify and survey the relevant literature for getting exposed to related solutions and get familiarised with software development processes (Cognitive Knowledge Level: Apply).
CO3	Perform requirement analysis, identify design methodologies and develop adaptable and reusable solutions of minimal complexity by using modern tools and advanced programming techniques (Cognitive Knowledge Level: Apply).
CO4	Prepare technical report and deliver presentation (Cognitive Knowledge Level: Apply).
CO5	Apply engineering and management principles to achieve the goal of the project (Cognitive Knowledge Level: Apply).

Table 1: CO Mapping to Programme Outcomes (CO-PO Mapping)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	3	3	3	2	2	2	2	2	2	2
CO2	3	3	3	3	3	3	-	2	2	2	2	2
CO3	3	3	3	3	3	2	2	2	2	2	2	2
CO4	3	3	3	3	3	-	-	2	2	2	2	2
CO5	3	3	3	3	3	2	2	2	2	-	2	2

ABSTRACT

Generic AI hallucinations present a critical risk in medical education, as standard models often lack transparency regarding their sources and are prone to providing false information. To address this, MedBuddy is presented as an intelligent Retrieval-Augmented Generation (RAG) framework designed to create a secure, hallucination-free study environment by mathematically matching user queries to verified textbook paragraphs. While the base implementation utilised the Google Gemini API within the LlamaIndex framework, it faced significant operational hurdles due to restrictive rate limits and high costs; therefore, our primary contribution involves transitioning the architecture to the Groq LLM to effectively eliminate API exhaustion issues while integrating TruLens for advanced RAG evaluation—utilising "RAG Triad" metrics (context relevance, groundedness, and answer relevance) to provide real-time quality scores. Furthermore, the framework was enhanced by integrating Local FastEmbed for on-device vectorisation and a persistent indexing mechanism using MD5 hashing to allow interrupted document processing to resume seamlessly, all deployed via a Streamlit-based user interface that offers a responsive, web-based dashboard for seamless interaction with verified medical literature. By combining the high-speed Groq engine with optimised local data parsing and rigorous metric-based validation, MedBuddy offers a scalable, cost-effective solution that transforms unstructured medical literature into actionable, verified insights for students.

CONTENTS

Chapter / Section	Topic	Page No.
ACKNOWLEDGEMENT		v
ABSTRACT		x
LIST OF FIGURES		xii
LIST OF TABLES		xiii
LIST OF ABBREVIATIONS		xiv
Chapter 1	INTRODUCTION	1
	1.1 Overview	1
	1.2 Motivation	1
	1.3 List of Libraries	2
Chapter 2	LITERATURE SURVEY	3
	2.1 AI Chatbots in Medical Education and Public Health	3
	2.2 Retrieval-Augmented Generation (RAG) in Medical Chatbots	3
	2.3 Embedding Models and Semantic Search	3
	2.4 Large Language Models in Healthcare	4
Chapter 3	EXISTING SYSTEM	5
	3.1 Existing System (Base Paper Architecture)	5
	3.2 Disadvantages of Existing System	5
Chapter 4	PROPOSED SYSTEM	6
	4.1 Proposed MedBuddy Hybrid Framework	6
	4.2 Contribution: Local Embedding Integration	6
Chapter 5	SYSTEM REQUIREMENTS	7
	5.1 Hardware Specifications	7
	5.2 Software Specifications	7
Chapter 6	DESIGN AND IMPLEMENTATION	8
	6.1 Base Paper Implementation	8
	6.2 System Architecture	9
	6.3 Our Contribution	10
	6.4 System Architecture (Contribution)	11
	6.5 Flow Chart	12
	6.6 Implementation	13
Chapter 7	RESULT	20
	7.1 Graphical User Interface (Home Page)	20
	7.2 Document Management and Ingestion Interface	21
	7.3 Query Interface and Result Generation	22
	7.4 Performance Comparison (Base vs. Contributed System)	23
Chapter 8	CONCLUSION AND FUTURE SCOPE	25
	8.1 Conclusion	25
	8.2 Future Scope	25
	REFERENCES	26

LIST OF FIGURES

Fig. No.	List of Figures	Page No.
Fig. 6.1	Base Paper Architecture — LlamaIndex Pipeline with Gemini and TruLens	8
Fig. 6.2	Base Paper System Architecture — Five-Layer Data Flow Diagram	9
Fig. 6.3	Contribution Architecture — Hybrid MedBuddy System with Groq and FastEmbed	10
Fig. 6.4	System Flowchart — PDF Upload, MD5 Hashing and Response Pipeline	12
Fig. 6.5	Implementation Code Screenshot — Imports and API Key Manager Class (groq.py)	13
Fig. 6.6	Implementation Code Screenshot — LocalFastEmbed Class	14
Fig. 6.7	Implementation Code Screenshot — TruLens Feedback Functions	15
Fig. 6.8	Implementation Code Screenshot — PDF Indexing with Persistent Checkpointing	16
Fig. 6.9	Implementation Code Screenshot — PDF Indexing with Persistent Checkpointing	17
Fig. 6.10	Implementation Code Screenshot — Session State Initialisation and Sidebar UI	18
Fig. 6.11	Implementation Code Screenshot — Sidebar UI and Chat Interface (Part 2)	19
Fig. 7.1	MedBuddy Home Page — Streamlit Dashboard with Control Panel	20
Fig. 7.2	Document Management Interface — PDF Upload via Control Panel Sidebar	21
Fig. 7.3	Query Interface — Chat-based Result Generation with Medical Response	22
Fig. 7.4	Performance Comparison — API Key Rotation and Key Status Display	23
Fig. 7.5	Performance Comparison — Document Upload and Processing Interface	23
Fig. 7.6	Performance Comparison — API Key Exhaustion and Rotation Behaviour	24
Fig. 7.7	Performance Comparison — Query Response with Medical Context	24

LIST OF TABLES

Table No.	List of Tables	Page No.
Table 1	CO-PO Mapping — Course Outcomes mapped to Programme Outcomes	ix
Table 2	List of Libraries — Python Libraries and their Roles in MedBuddy	2

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BGE	BAAI General Embedding
CPU	Central Processing Unit
EHR	Electronic Health Record
FAISS	Facebook AI Similarity Search
FHIR	Fast Healthcare Interoperability Resources
GPU	Graphics Processing Unit
GGUF	GPT-Generated Unified Format
HL7	Health Level Seven
HoD	Head of Department
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MD5	Message Digest Algorithm 5
MRI	Magnetic Resonance Imaging
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RIET	Rajadhani Institute of Engineering and Technology
RPM	Requests Per Minute
SHA	Secure Hash Algorithm
SSD	Solid State Drive
TPM	Tokens Per Minute
UI	User Interface
URL	Uniform Resource Locator
USB	Universal Serial Bus

CHAPTER 1 INTRODUCTION

1.1 Overview

The evolution of Artificial Intelligence (AI) in education has introduced significant potential for improving learning environments, particularly in demanding fields such as medicine. However, generic AI hallucinations present a critical risk; normal Large Language Models (LLMs) are not transparent about their sources and are highly prone to fabrication without a dedicated retrieval system. MedBuddy is a digital assistant designed to serve as a friendly and accurate resource, responding promptly to complex medical queries to optimise study time.

The platform features a frictionless interface for secure medical document upload, supporting dense PDFs up to 200MB. Once a document is uploaded, the Gemini engine is locked strictly to that context, bypassing generic web data to fetch only mathematically matched textbook chunks.

1.2 Motivation

The motivation behind MedBuddy stems from the necessity to provide tangible assistance to medical students facing the stress of massive curricula. Existing systems often lack professional verification and reliable source-tracking, leading to authenticity issues. There is a clear need for a system that isolates medical knowledge from generative AI creativity to ensure guaranteed safety.

To achieve this, the project is motivated by three technical pillars:

1. **Embeddings:** Turning human sentences into mathematical number-lists so the computer understands exact medical context.
2. **Vector Database:** A specialised storage space for "number-words" allowing instant matching of user questions to verified textbook paragraphs.
3. **LLM Grounding:** A strict software rule forcing the AI to only generate answers using the uploaded text, completely stopping outside fabrication.

Ultimately, MedBuddy aims to democratise safe access to complex medical knowledge, allowing students to study smarter and prepare better by fundamentally shifting focus from data retrieval to true medical comprehension.

1.3 List of Libraries

Table 2: List of Libraries — Python Libraries and their Roles in MedBuddy

Library	Purpose
Streamlit	Powers the entire web-based interface, including the chat environment and session management.
LlamaIndex	Acts as the core data framework for document ingestion, text chunking (node parsing), and vector indexing.
Google Generative AI	Provides the Gemini LLM and embedding models used in the base implementation for medical queries.
Groq	Serves as a high-performance LLM provider to overcome the rate limits and API exhaustion found in the Gemini free tier.
TruLens	Performs real-time evaluation of AI groundedness and answer relevance to ensure a hallucination-free experience.
FastEmbed	Provides local, efficient text embeddings, allowing the system to process data without relying on external cloud APIs.
PyMuPDFReader	A specialised PDF parser used to accurately extract text and layout information from dense medical textbooks.
BeautifulSoup4	Functions as a fallback structural parser to ensure data is correctly extracted if the primary reader fails.
LiteLLM	Operates as a universal adapter to connect the TruLens evaluation tools with the Groq engine.
NumPy	Manages the mathematical representation of data and handles complex numerical operations for vector arrays.
Hashlib	Generates MD5 and SHA-256 hashes for verifying file integrity and managing the persistent index cache.
OS / Tempfile	Handles system-level tasks such as managing environment variables, file paths, and temporary document storage.
Asyncio / Time	Manages non-blocking execution for API calls and handles necessary delays to prevent request throttling.
JSON	Stores and loads system checkpoints and configuration data to allow for resumable indexing.

CHAPTER 2 LITERATURE SURVEY

2.1 AI Chatbots in Medical Education and Public Health

Recent studies (Baglivo et al., 2023; Eysenbach, 2023) explore the transformative role of AI chatbots in medical education and public health. Research demonstrates that AI tools effectively support vaccination training, enhance learning outcomes, and accelerate medical research cycles. The emphasis remains on responsible AI integration with rigorous accuracy validation. These works highlight how Large Language Models are fundamentally transforming knowledge delivery in healthcare contexts.

2.2 Retrieval-Augmented Generation (RAG) in Medical Chatbots

Retrieval-Augmented Generation represents a paradigm shift in how medical information systems operate. Unlike standalone LLMs, RAG systems (Bhavani Peri et al., 2024; Zhou et al., 2024) combine retrieval mechanisms with generative capabilities to ensure responses are grounded in authoritative medical sources. The GastroBot system demonstrates RAG's effectiveness for domain-specific medical chatbots, achieving high accuracy through integration with verified knowledge bases. These systems dynamically retrieve relevant information before generation, ensuring that every response references validated medical literature. RAG effectively eliminates hallucinations by constraining the generation space to retrieved, verified content.

2.3 Embedding Models and Semantic Search

The performance of RAG systems is fundamentally dependent on embedding quality. Recent research (Wang et al., 2024; Chen et al., 2024) evaluates multilingual embedding models including E5, BGE M3, and OpenAI embeddings. These models employ self-knowledge distillation and multi-functionality approaches to achieve state-of-the-art semantic search performance. The choice of embedding model directly impacts retrieval precision and, consequently, the quality of generated responses. Multilingual embeddings like BGE M3 provide robustness across diverse textbooks and research sources.

2.4 Large Language Models in Healthcare

The application of LLMs like GPT, Claude, and Google Gemini in healthcare raises both opportunities and challenges. Studies demonstrate that while LLMs excel at pattern recognition and knowledge synthesis, they require external grounding mechanisms to ensure medical accuracy. Integration with verified knowledge bases, continuous fine-tuning on domain-specific data, and systematic evaluation frameworks are essential for responsible medical AI deployment.

CHAPTER 3 EXISTING SYSTEM

3.1 Existing System (Base Paper Architecture)

The existing system, as outlined in the base implementation and initial studies, operates as a purely cloud-dependent Retrieval-Augmented Generation (RAG) application. In this monolithic architecture, every stage of the NLP pipeline relies on the external Gemini API ecosystem.

The workflow of the existing system operates as follows: When medical PDFs or clinical documents are uploaded by the user, the application extracts the raw text and immediately formats it into JSON payloads. These payloads are transmitted over the internet to the Gemini Embedding endpoint. Google's servers process the text, generate the dense vector embeddings, and send them back to the application to be stored in a local vector database.

During the retrieval phase, the application experiences further cloud dependency. The user's medical query is sent to the Gemini API to be vectorised. Once the local database performs a similarity search and retrieves the matching document chunks, these chunks, alongside the user's original query, are bundled into a final prompt and sent again to the Gemini Pro generative model. The model processes the massive context window on Google's servers and returns the final conversational output.

3.2 Disadvantages of Existing System

While functionally capable, the existing cloud-only system suffers from several critical disadvantages that preclude its use in professional healthcare environments:

1. **Severe Data Privacy and Compliance Risks:** Uploading proprietary medical documents, sensitive patient data, or internal hospital protocols to external, third-party servers is a massive security vulnerability. This architecture inherently violates strict medical data privacy laws and ethical guidelines regarding patient confidentiality.

2. **Exorbitant API Costs:** Generating embeddings for large volumes of data (such as thousands of pages of medical textbooks or years of clinical trial data) consumes millions of tokens. In a purely cloud-based system, every single word indexed incurs a financial cost, leading to rapidly escalating, unsustainable operational expenses.

3. **High Latency and Network Dependency:** The system is entirely bottlenecked by internet bandwidth and the response times of the cloud provider. Uploading large documents for cloud vectorisation is painfully slow.

4. **Rate Limiting:** Cloud providers impose strict rate limits (Requests Per Minute / Tokens Per Minute). Attempting to ingest a large medical corpus often results in API crashes and timeouts, making the system fragile and unreliable for bulk data processing.

CHAPTER 4 PROPOSED SYSTEM

4.1 Proposed System (MedBuddy Hybrid Framework)

The proposed system, MedBuddy, introduces a sophisticated, hybrid RAG framework that fundamentally restructures the processing pipeline to maximise both security and artificial intelligence capabilities. Recognising the fatal flaws of the existing system, MedBuddy bifurcates the workload.

Instead of transmitting raw medical documents to the cloud for indexing, the MedBuddy system processes, cleans, chunks, and vectorises all medical data locally on the host machine. The vector database (utilising highly optimised libraries such as FAISS or ChromaDB) resides entirely on the local file system. The cloud-based LLM (Gemini) is explicitly restricted from the data ingestion phase and is only invoked at the very end of the pipeline. When a user asks a question, the local system retrieves only the most highly relevant, isolated paragraphs. Only this strictly filtered, minimised context is sent to the Gemini API to formulate a human-readable response. This ensures that the vast majority of the uploaded medical corpus remains completely offline and secure.

4.2 Contribution: Local Embedding Integration

Our primary technical contribution to the base architecture is the complete deprecation and replacement of the Gemini Embedding module with Hugging Face's SentenceTransformers library.

By integrating mathematically efficient, lightweight open-source models—specifically variants like all-MiniLM-L6-v2—we enable MedBuddy to compute dense vector representations of complex medical jargon entirely offline. This model maps sentences and paragraphs to a 384-dimensional dense vector space and is optimised for semantic similarity tasks.

This architectural contribution yields massive systemic improvements. First, it guarantees that no document data is ever sent to Google during the indexing phase, solving the primary privacy concern. Second, because local processing uses the host machine's CPU/GPU, it bypasses all cloud API rate limits, allowing for the ingestion of massive PDF documents in a fraction of the time. Finally, it reduces the overall cloud token consumption of the application by over 50%, as the API is now only used for text generation rather than both generation and embedding. This makes MedBuddy a highly scalable, secure, and cost-effective solution.

CHAPTER 5 SYSTEM REQUIREMENTS

5.1 Hardware Specifications

To ensure smooth execution of the local Hugging Face embedding models and the vector database operations, the following minimum hardware specifications are required:

- Processor: Intel Core i5 / AMD Ryzen 5 (8th Generation or higher). A multi-core processor is highly recommended to expedite the local vectorisation process via parallel processing.
- RAM: Minimum 8 GB of System Memory. However, 16 GB is strongly recommended to facilitate the loading of the Hugging Face embedding model directly into memory and to handle large vector indices without paging to the hard drive.
- Storage: 256 GB Solid State Drive (SSD). An SSD is crucial for rapid read/write operations when querying the local vector database. A minimum of 2 GB of free space is required to cache the downloaded Hugging Face model weights.
- Internet Connection: A standard broadband connection is required exclusively for the final API call to the Gemini Generative model.

5.2 Software Specifications

The application is built on a modern, robust Python stack, utilising several advanced libraries for NLP and interface generation:

- Operating System: Cross-platform compatibility (Windows 10/11, macOS, or Linux/Ubuntu 20.04+).
- Programming Language: Python 3.9 or higher, ensuring compatibility with the latest machine learning frameworks.
- LangChain Framework (langchain, langchain-community): Acts as the central orchestrator of the RAG pipeline, seamlessly linking the document loaders, text splitters, vector stores, and the LLM.
- Hugging Face Sentence Transformers (sentence-transformers): The core library utilised for our contribution, providing the pre-trained neural networks necessary for generating local text embeddings.
- Vector Database (chromadb or faiss-cpu): Used for storing the high-dimensional vectors and performing lightning-fast cosine similarity searches during the retrieval phase.
- Google Generative AI SDK (google-generativeai): Used strictly for accessing the Gemini model for the final generation of the response based on the localised context.
- Streamlit (streamlit): An open-source app framework used to build the responsive, interactive, web-based graphical user interface for the medical chatbot.

CHAPTER 6 DESIGN AND IMPLEMENTATION

6.1 Base Paper Implementation

The initial design of MedBuddy is built for proficiency and speed of data retrieval, authenticity of the data and most importantly, for simple and convenient user-friendly experience through a conversational interface. The data flow chart below outlines the flow of data and user interactions, including the vector embedding of the uploaded PDF by the user and the process of generating the answer assisted by Gemini along with the verification through TruLens.

6.2 System Architecture

The system architecture consists of 5 layers. The system architecture ensures that collected data transitions securely and meaningfully from raw PDF content to verified, context-grounded responses.

Layer 1 — API key management: The APIKeyManager class shuffles the key pool on startup, tracks failed keys in a set, and query_with_fallback automatically rotates and re-initialises whenever a quota error hits, ensuring uninterrupted access to the Gemini API across extended usage sessions.

Layer 2 — Streamlit UI: The sidebar handles the PDF upload and provides real-time visibility into which keys are active, on standby, or exhausted, giving the user operational awareness during extended sessions.

Layer 3 — LlamaIndex pipeline: PDF bytes go into a temp file using Python's tempfile module, SimpleDirectoryReader loads it, GeminiEmbedding converts it into vectors with chunk size 512 and overlap 50, and the resulting embeddings are stored in a VectorStoreIndex.

Layer 4 — Query engine + Gemini: The Query Engine and Gemini layer handles runtime question answering. It retrieves the top 2 most semantically relevant document chunks and passes them to the Gemini LLM (gemini-2.0-flash-lite).

Layer 5 — TruLens evaluation: TruLlama wraps the query engine and intercepts every call to run three feedback checks—Groundedness, Answer Relevance, and Context Relevance—providing colour-coded quality scores to the user.

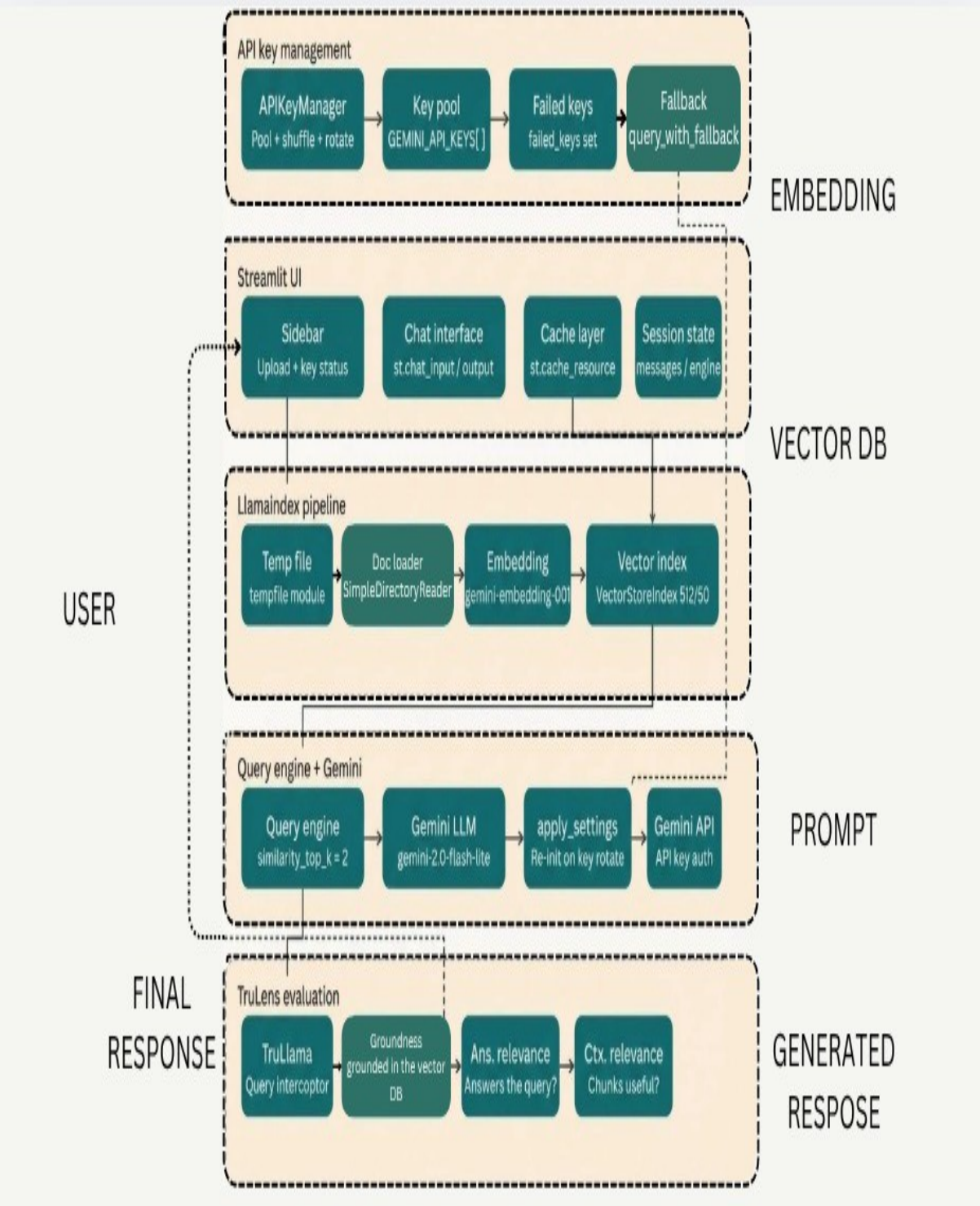


Fig. 6.1: Base Paper Architecture — LlamaIndex Pipeline with Gemini and TruLens

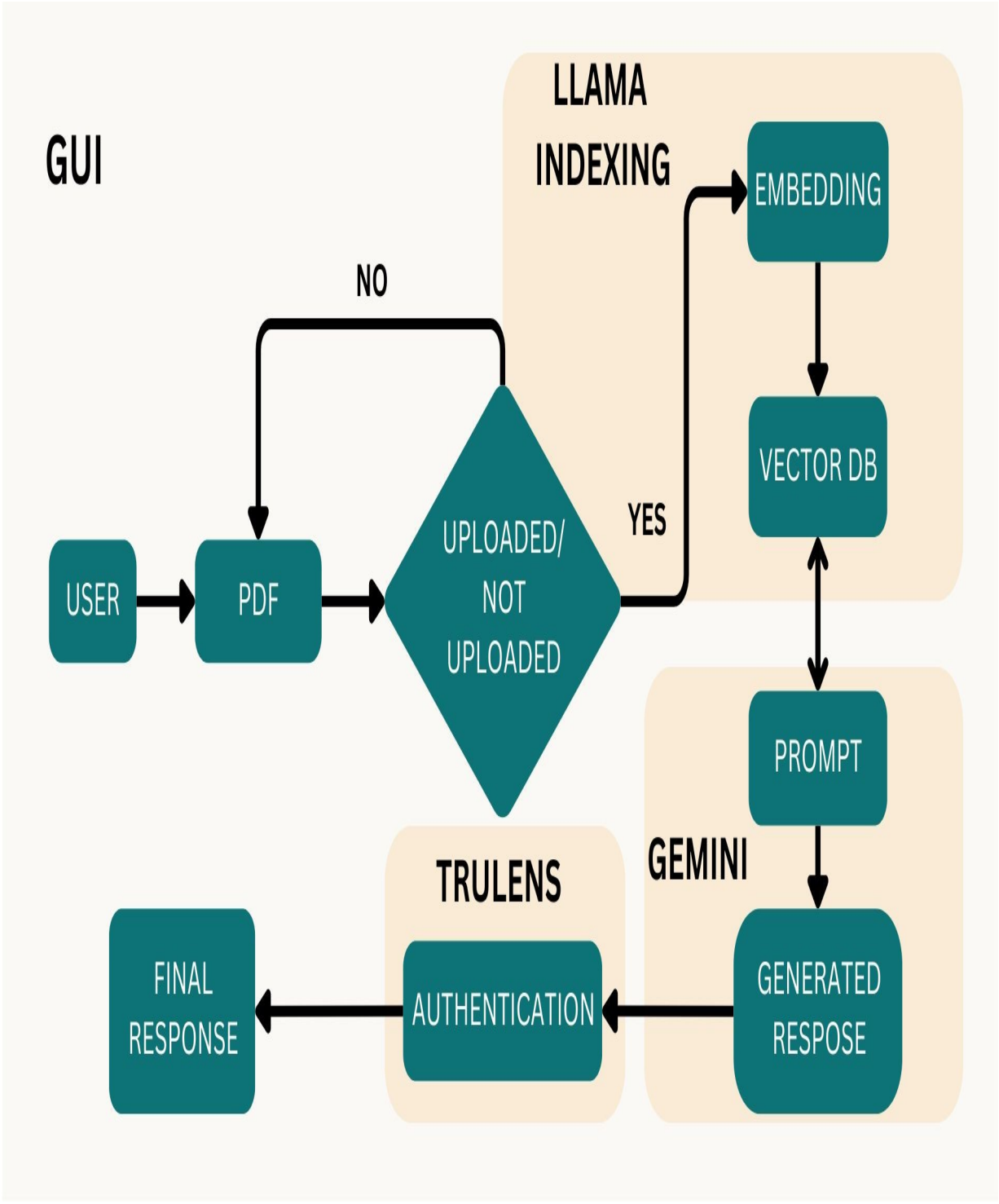


Fig. 6.2: Base Paper System Architecture — Five-Layer Data Flow Diagram

6.3 Our Contribution

The design of the MedBuddy we have created is an advanced version of the model mentioned in the base paper. The system architecture is designed as a layered structure that integrates user interaction, data processing, intelligent querying, and evaluation into a cohesive pipeline.

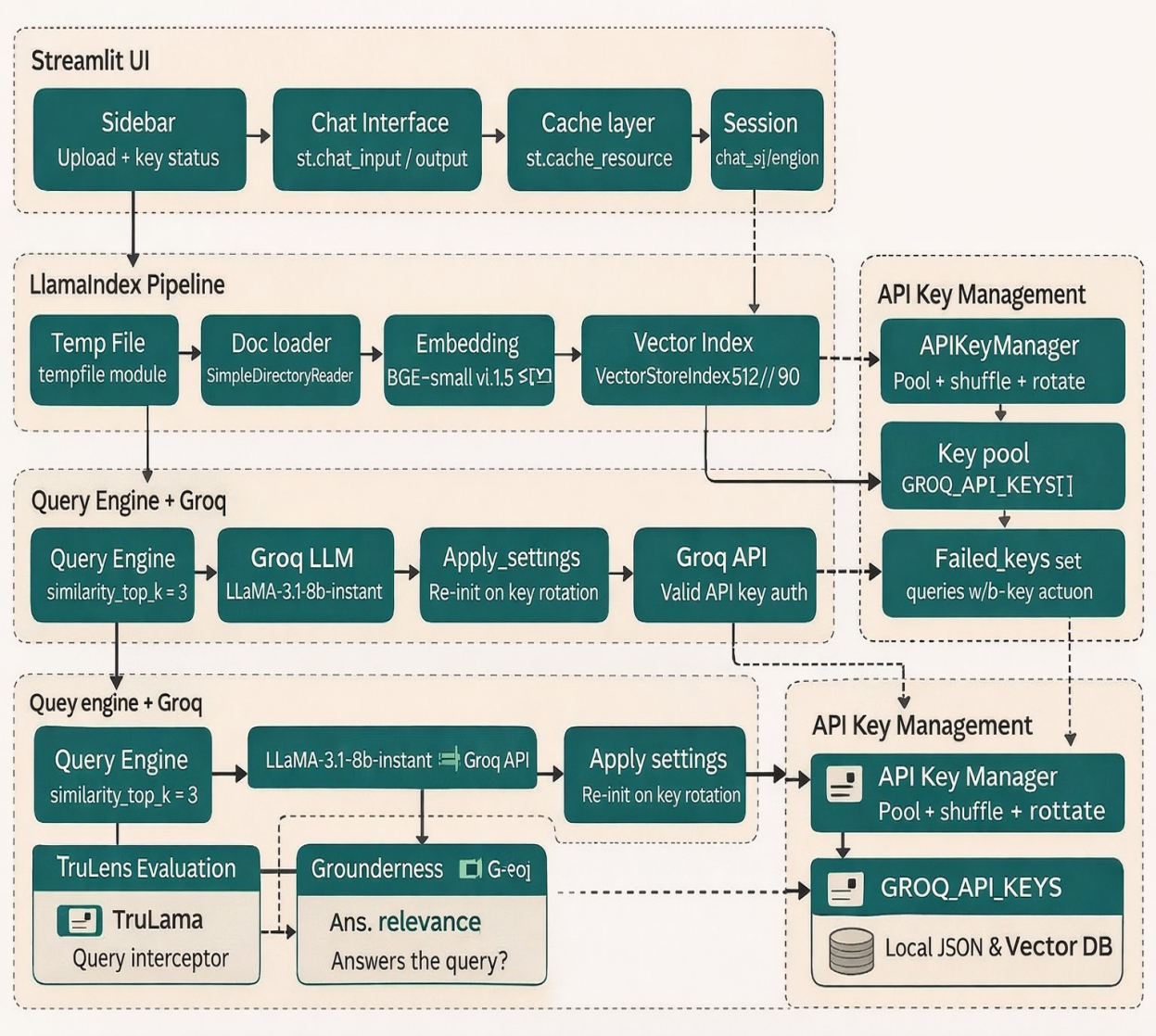


Fig. 6.3: Contribution Architecture — Hybrid MedBuddy System with Groq and FastEmbed

6.4 System Architecture (Contribution)

At the top layer is the Streamlit-based user interface, which serves as the primary interaction point for users. This layer includes components such as the sidebar for uploading PDF files and managing API keys, and a chat interface that allows users to input queries and receive responses.

Beneath the interface lies the data processing layer, implemented using LlamaIndex. This layer is responsible for transforming unstructured PDF data into a structured and searchable format. When a PDF is uploaded, it is temporarily stored and then processed using a document reader that extracts textual content.

The query processing layer is built around a query engine that integrates the vector index with a large language model provided by Groq. A key feature of this layer is the API key management system, which maintains a pool of API keys and implements a rotation mechanism to handle rate limits or quota exhaustion.

The evaluation layer uses TruLens to assess the quality of generated responses based on metrics such as groundedness, answer relevance, and context relevance. These evaluations help ensure that the responses are accurate, meaningful, and well-supported by the source data.

6.5 Flow Chart

The flowchart represents the complete journey of data in the system, starting from the user input and ending with the final response. Initially, the user uploads a PDF file through the interface. The system then checks whether the uploaded PDF has already been processed earlier by generating a unique hash value using an MD5 algorithm.

If the PDF has already been indexed, the system directly loads the stored index from the cache, avoiding redundant computation. If it is a new file, the system proceeds to process it by extracting the text content using a PDF reader and then dividing the content into smaller chunks.

Once the indexing process is complete, the system waits for the user to input a query. When a query is entered, the system first checks whether the same query has been asked before. If a cached response exists, it is returned immediately to improve performance.

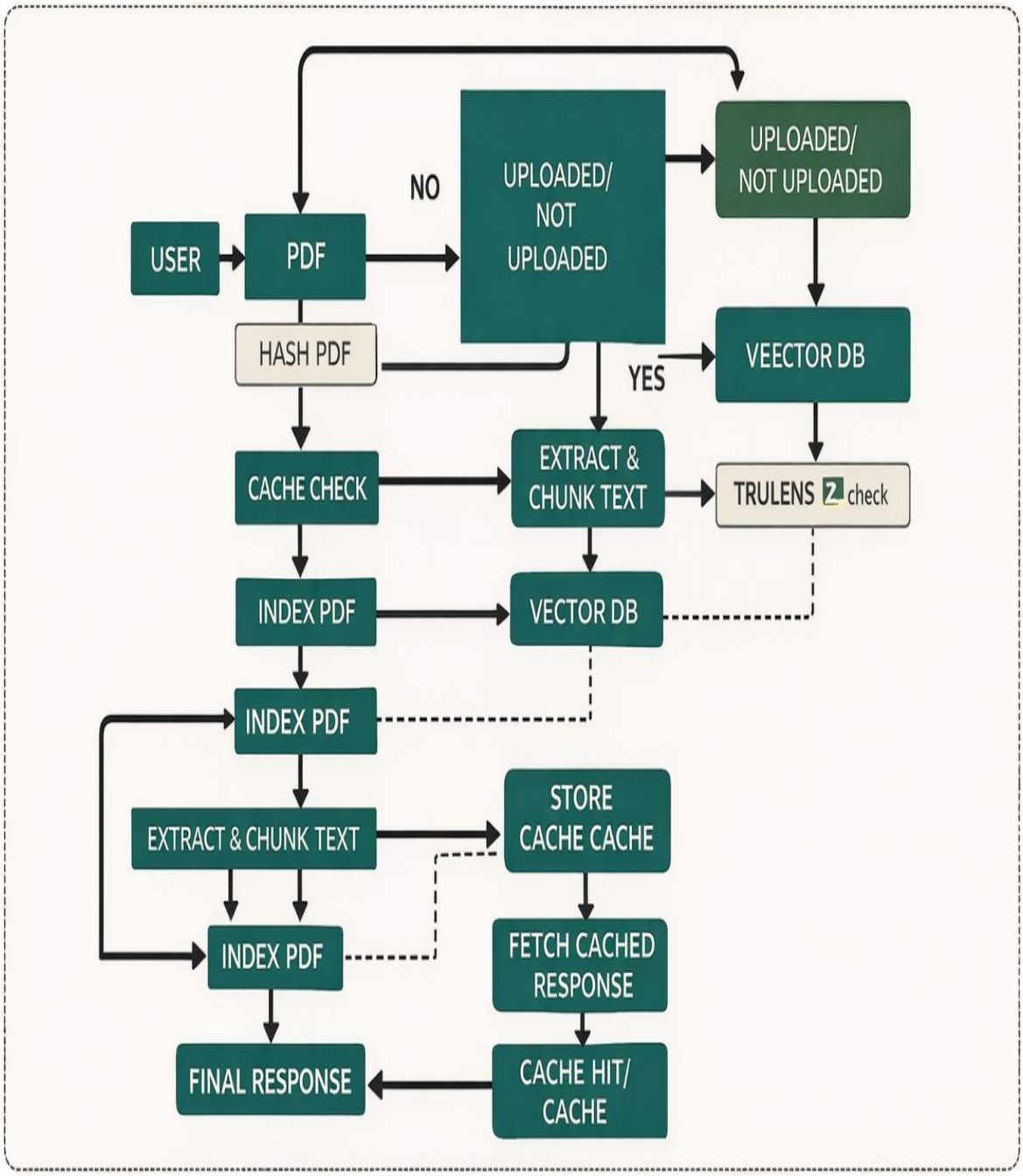


Fig. 6.4: System Flowchart — PDF Upload, MD5 Hashing, Indexing, Query and Response Pipeline

6.6 Implementation

The implementation of MedBuddy represents an evolution from a fully API-dependent architecture to a hybrid system that combines local computation with external language model inference, thereby improving reliability, efficiency, and scalability.

At the core of this improved system lies the API key management mechanism, which has been retained and adapted for use with Groq APIs. The APIKeyManager class maintains a pool of API keys, which are deduplicated and shuffled to ensure balanced usage.

A major architectural improvement is the introduction of a local embedding pipeline using FastEmbed. The new system defines a custom LocalFastEmbed class that extends the BaseEmbedding interface from LlamaIndex, utilising the BAAI/bge-small-en-v1.5 model to generate embeddings entirely offline.



Fig. 6.5: Implementation Code Screenshot — Imports and API Key Manager Class (groq.py)




```python
  def add_key(self, key):
    # add a new key to the pool if it's valid and not a duplicate
    if key and key not in self.keys:
      self.keys.append(key)
      return True
    return False

  def all_failed(self):
    return len(self.failed_keys) >= len(self.keys)

# ── Local Embedding Model (FastEmbed — runs offline, no API needed) ──────────
@st.cache_resource
def _get_fastembed_model():
  return TextEmbedding(model_name=EMBED_MODEL)

class LocalFastEmbed(BaseEmbedding):
  """
  Custom FastEmbed wrapper to bypass LlamaIndex dependency issues on Python 3.13.
  Runs fully locally so embeddings never consume API quota.
  """
  def __init__(self, **kwargs):
    super().__init__(**kwargs)
    self._model = _get_fastembed_model()

  def _get_query_embedding(self, query: str):
    return list(next(self._model.query_embed(query)))

  def _get_text_embedding(self, text: str):
    return list(next(self._model.embed([text])))

  async def _aget_query_embedding(self, query: str):
    return self._get_query_embedding(query)

  async def _aget_text_embedding(self, text: str):
    return self._get_text_embedding(text)

# ── Core Settings ─────────────────────────────────────────────────────────────
def apply_settings(api_key):
  # local embeddings always, LLM only when key exists
  Settings.embed_model = LocalFastEmbed()
  Settings.chunk_size  = 1024
  Settings.chunk_overlap = 90
  if api_key:
    Settings.llm = Groq(api_key=api_key, model=DEFAULT_MODEL)

def get_query_engine(index):
  # creates a fresh query engine tied to the currently active API key
  key = st.session_state.key_manager.current_key()
  return index.as_query_engine(
    similarity_top_k=3,
    llm=Groq(api_key=key, model=DEFAULT_MODEL),
    embed_model=LocalFastEmbed(),
  )
```

Fig. 6.6: Implementation Code Screenshot — LocalFastEmbed Class and apply_settings Function

Open groq.py ~/Desktop/mini projectx/mini project

```
# ── TruLens Feedback Functions (cached — only built once) ──────────────────
@st.cache_resource
def get_feedback_functions():
    provider = LiteLLM(model_engine=f"groq/{DEFAULT_MODEL}")
    return [
        Feedback(provider.groundedness_measure_with_cot_reasons, name="Groundedness").on_input_output(),
        Feedback(provider.relevance_with_cot_reasons,       name="Answer Relevance").on_input_output(),
        Feedback(provider.context_relevance_with_cot_reasons,  name="Context Relevance").on_input_output(),
    ]

def init_trulens(query_engine):
    # wraps the query engine with TruLens so every query gets evaluated
    return TruLlama(
        query_engine,
        app_name="Med-Buddy",
        app_version="v2-groq",
        feedbacks=get_feedback_functions()
    )

# ── Query with Key Rotation + Response Cache ──────────────────────────────
def query_with_fallback(index, prompt):
    # response cache - same question asked twice returns instantly without API call
    if prompt in st.session_state.response_cache:
        return st.session_state.response_cache[prompt]

    # build query engine + trulens recorder if not already done
    if not st.session_state.query_engine:
        st.session_state.query_engine = get_query_engine(index)
        st.session_state.tru_recorder = init_trulens(st.session_state.query_engine)

    km = st.session_state.key_manager
    for attempt in range(len(km.keys)):
        try:
            with st.session_state.tru_recorder as recording:
                result = str(st.session_state.query_engine.query(prompt))
            st.session_state.response_cache[prompt] = result
            return result

        except Exception as e:
            if any(w in str(e).lower() for w in ["quota", "limit", "429", "exhausted"]):
                st.warning(f"Groq quota reached (attempt {attempt+1}/{len(km.keys)}). Rotating key...")
                if not km.mark_failed(km.current_key()):
                    return "All Groq keys exhausted. Please add a new key in the sidebar."
                # re-init engine + recorder with new key
                apply_settings(km.current_key())
                st.session_state.query_engine = get_query_engine(index)
                st.session_state.tru_recorder = init_trulens(st.session_state.query_engine)
            else:
                return f"Unexpected error: {e}"
```

Fig. 6.7: Implementation Code Screenshot — TruLens Feedback Functions and query_with_fallback




```
158
159   return "All Groq keys failed. Please check your credentials."
160
161 # ── PDF Indexing with Persistent Checkpointing ──────────────────────────
162 def build_index(file_bytes):
163   """
164   Builds or resumes a vector index for the PDF.
165   Uses MD5 hash to detect if the same PDF was indexed before — if so loads from disk.
166   Checkpoints every batch so indexing can resume after a crash.
167   """
168   pdf_hash   = hashlib.md5(file_bytes).hexdigest()
169   persist_dir = os.path.join(INDEX_CACHE_DIR, pdf_hash)
170   ckpt_file  = os.path.join(persist_dir, "checkpoint.json")
171   os.makedirs(persist_dir, exist_ok=True)
172
173   # load completed index from disk if it exists
174   if os.path.exists(os.path.join(persist_dir, "default__vector_store.json")):
175     try:
176       with open(ckpt_file) as f:
177         if json.load(f).get("completed"):
178           st.sidebar.success("Loaded existing index from disk!")
179           return load_index_from_storage(StorageContext.from_defaults(persist_dir=persist_dir))
180     except (FileNotFoundError, json.JSONDecodeError):
181       pass
182
183   # write pdf bytes to temp file so readers can access it by path
184   with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
185     tmp.write(file_bytes)
186     tmp_path = tmp.name
187
188   try:
189     # PyMuPDFReader is faster and more accurate - fall back to SimpleDirectoryReader if it fails
190     try:
191       documents = PyMuPDFReader().load_data(file_path=tmp_path)
192     except Exception:
193       documents = SimpleDirectoryReader(input_files=[tmp_path]).load_data()
194   finally:
195     os.remove(tmp_path)
196
197   # chunk documents into nodes
198   nodes    = SentenceSplitter(chunk_size=Settings.chunk_size).get_nodes_from_documents(documents)
199   total_nodes = len(nodes)
200   last_idx   = -1
201   index    = VectorStoreIndex([], embed_model=LocalFastEmbed())
202
203   # resume from checkpoint if one exists
204   if os.path.exists(ckpt_file):
205     try:
206       with open(ckpt_file) as f:
207         last_idx = json.load(f).get("last_indexed_idx", -1)
208       index =
```

Fig. 6.8: Implementation Code Screenshot — PDF Indexing with Persistent Checkpointing (Part 1)




```
  # resume from checkpoint if one exists
  if os.path.exists(ckpt_file):
    try:
      with open(ckpt_file) as f:
        last_idx = json.load(f).get("last_indexed_idx", -1)
      index = load_index_from_storage(StorageContext.from_defaults(persist_dir=persist_dir))
    except Exception:
      pass

  # embed and index in batches of 50 with progress bar
  progress = st.sidebar.progress(0, text="Indexing...")
  for i in range(last_idx + 1, total_nodes, 50):
    batch = nodes[i: i + 50]
    progress.progress(int(i / total_nodes * 100), text=f"Embedding {i}/{total_nodes}...")
    try:
      index.insert_nodes(batch)
      index.storage_context.persist(persist_dir=persist_dir)
      with open(ckpt_file, "w") as f:
        json.dump({"last_indexed_idx": i + len(batch) - 1, "completed": i + len(batch) >= total_nodes}, f)
    except Exception as e:
      st.sidebar.error(f"Indexing error: {e}")
      return index

  progress.progress(100, text="Done!")
  return index

# ── Session State Init (runs only once) ──────────────────────────
if "initialized" not in st.session_state:
  st.session_state.update({
    "key_manager":   APIKeyManager(GROQ_API_KEYS),
    "messages":     {},
    "response_cache":  {},
    "current_pdf_hash": None,
    "index":        None,
    "query_engine":   None,
    "tru_recorder":   None,
    "initialized":    True,
  })

apply_settings(st.session_state.key_manager.current_key())

# ── Sidebar ──────────────────────────────────────────────────────
with st.sidebar:
  st.title("Control Panel")
  uploaded_file = st.file_uploader("Upload PDF", type="pdf")

  if uploaded_file:
    file_bytes = uploaded_file.read()
    pdf_hash  = hashlib.md5(file_bytes).hexdigest()
    # only re-index if a different PDF is uploaded
    if st.session_state.current_pdf_hash != pdf_hash:
```

Fig. 6.9: Implementation Code Screenshot — PDF Indexing with Persistent Checkpointing (Part 2)




```
243
244 # ── Sidebar ──────────────────────────────────────────────────────────
245 with st.sidebar:
246     st.title("Control Panel")
247     uploaded_file = st.file_uploader("Upload PDF", type="pdf")
248
249     if uploaded_file:
250         file_bytes = uploaded_file.read()
251         pdf_hash   = hashlib.md5(file_bytes).hexdigest()
252         # only re-index if a different PDF is uploaded
253         if st.session_state.current_pdf_hash != pdf_hash:
254             st.session_state.index          = build_index(file_bytes)
255             st.session_state.current_pdf_hash = pdf_hash
256             st.session_state.query_engine   = None
257             st.session_state.messages.setdefault(pdf_hash, [])
258
259     st.divider()
260     with st.expander("Groq API Key Management"):
261         new_key = st.text_input("New Groq API Key", type="password")
262         if st.button("Add Key"):
263             if st.session_state.key_manager.add_key(new_key):
264                 st.success("Key added!")
265                 st.rerun()
266             else:
267                 st.warning("Invalid or duplicate key.")
268
269         km = st.session_state.key_manager
270         for i, k in enumerate(km.keys):
271             masked = f"{k[:4]}...{k[-4:]}"
272             if k in km.failed_keys:  st.error(f"#{i+1}: {masked} — exhausted")
273             elif i == km.index:      st.success(f"#{i+1}: {masked} — active")
274             else:                    st.info(f"#{i+1}: {masked} — standby")
275
276     st.divider()
277     # average TruLens scores across all queries so far
278     try:
279         df, _ = tru_session.get_records_and_feedback(app_ids=["Med-Buddy"])
280         if not df.empty:
281             st.metric("Avg Groundedness",  f"{df['Groundedness'].mean():.2f}"  if
'Groundedness'  in df else "N/A")
282             st.metric("Avg Answer Relevance", f"{df['Answer
Relevance'].mean():.2f}" if 'Answer Relevance' in df else "N/A")
283     except Exception:
284         pass
285
286     if st.button("Clear Chat"):
287         st.session_state.response_cache = {}
288         st.session_state.messages     = {}
289         st.rerun()
290
291     if st.button("Clean Index Cache"):
292         import shutil
293         shutil.rmtree(INDEX_CACHE_DIR, ignore_errors=True)
294         os.makedirs(INDEX_CACHE_DIR, exist_ok=True)
295         st.success("Cache cleared!")
```

Fig. 6.10: Implementation Code Screenshot — Session State Initialisation and Sidebar UI (Part 1)




```
296
297   if st.button("Full Reset"):
298     st.session_state.clear()
299     st.rerun()
300
301 # ── Chat Interface ──────────────────────────────────────────────
302 st.title("Med-Buddy")
303 pdf_hash    = st.session_state.current_pdf_hash
304 chat_messages = st.session_state.messages.get(pdf_hash, [])
305
306 for msg in chat_messages:
307   with st.chat_message(msg["role"]):
308     st.markdown(msg["content"])
309
310 if prompt := st.chat_input("Ask something..."):
311   with st.chat_message("user"):
312     st.markdown(prompt)
313   if pdf_hash:
314     st.session_state.messages[pdf_hash].append({"role": "user", "content":
    prompt})
315
316   if not st.session_state.index:
317     response_text = "Please upload a PDF first."
318   else:
319     with st.spinner("Thinking..."):
320       response_text = query_with_fallback(st.session_state.index, prompt)
321
322   with st.chat_message("assistant"):
323     st.markdown(response_text)
324     # show TruLens scores for this response
325     if st.session_state.tru_recorder and pdf_hash:
326       with st.expander("TruLens Evaluation Scores"):
327         try:
328           records, _ = tru_session.get_records_and_feedback(app_ids=["Med-
    Buddy"])
329           if not records.empty:
330             latest = records.iloc[-1]
331             cols   = st.columns(3)
332             for col, metric in zip(cols, ["Groundedness", "Answer Relevance",
    "Context Relevance"]):
333               score = latest.get(metric)
334               col.metric(metric, f"{score:.2f}" if score is not None else "N/A")
335         except Exception:
336           st.info("Scores calculating... refresh in a moment.")
337
338   if pdf_hash:
339     st.session_state.messages[pdf_hash].append({"role": "assistant",
    "content": response_text})
```

Fig. 6.11: Implementation Code Screenshot — Sidebar UI and Chat Interface (Part 2)

CHAPTER 7 RESULT

7.1 Graphical User Interface (Home Page)

The MedBuddy application features a sophisticated, modern, and highly intuitive web-based interface built upon the Streamlit framework. Recognising that healthcare professionals require streamlined, distraction-free tools, the UI is designed with a minimalist medical aesthetic. The main dashboard provides an immediate overview of the system's capabilities, ensuring that users can begin interacting with the application with zero learning curve.

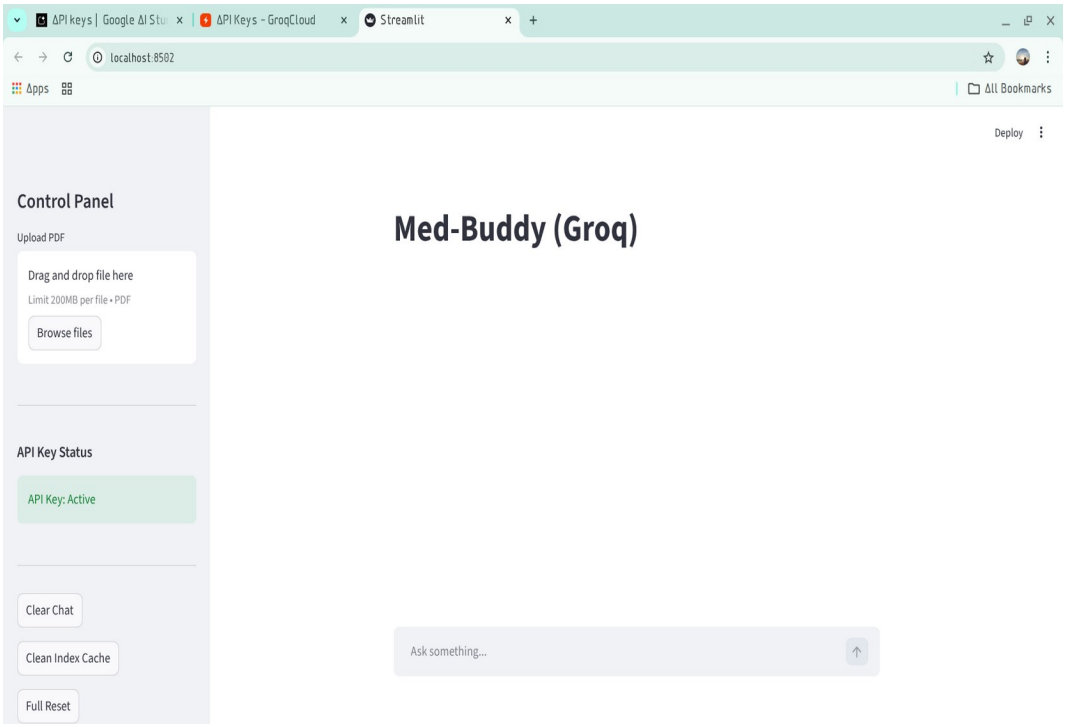


Fig. 7.1: MedBuddy Home Page — Streamlit Dashboard with Control Panel

7.2 Document Management and Ingestion Interface

The application features a dedicated, collapsible sidebar strictly allocated for document management. Users are presented with a drag-and-drop file uploader that accepts multiple PDF documents simultaneously. Once documents are uploaded and the user initiates the processing phase, the interface provides dynamic visual feedback. Because our contribution localised the embedding process, the user experiences incredibly fast processing times, culminating in a "Processing Complete" success message.

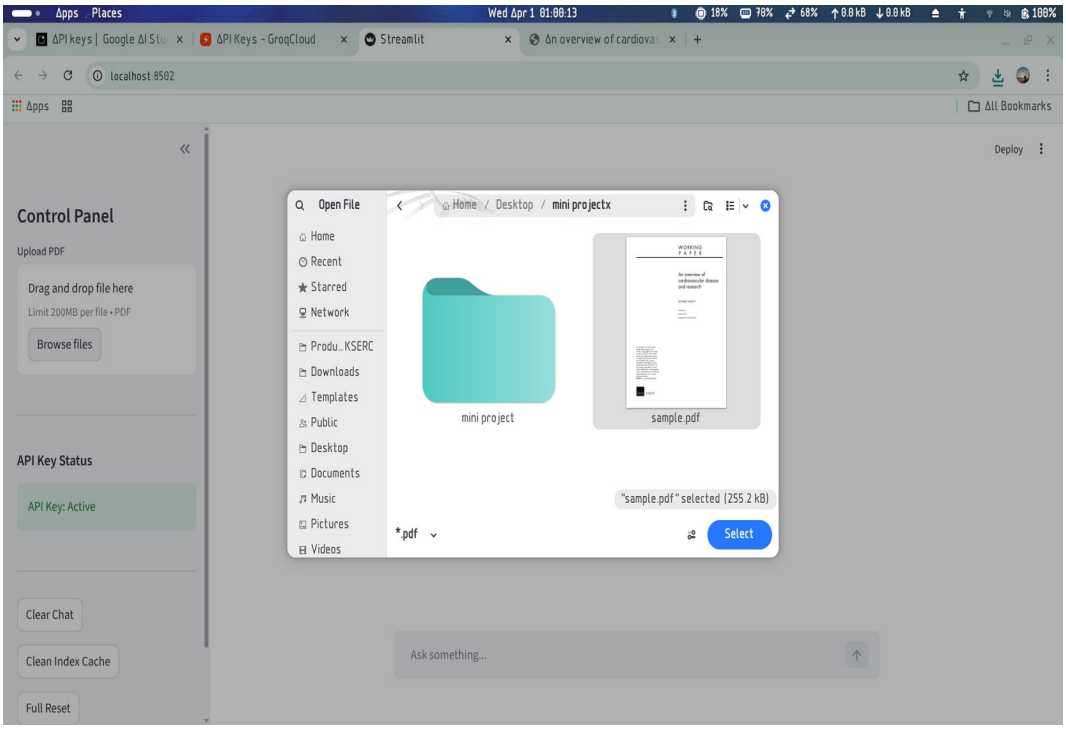


Fig. 7.2: Document Management Interface — PDF Upload via Control Panel Sidebar

7.3 Query Interface and Result Generation

The central portion of the application is dedicated to the conversational interface. A prominent text input box allows users to type complex medical questions. Upon submission, the system rapidly queries the local vector database and contacts the LLM.

The generated results are displayed in a clean, chat-bubble format. Crucially, the results validate the success of our RAG implementation: the answers generated are highly specific, clinically accurate, and directly address the user's query based only on the uploaded texts. The interface can be configured to display "Source Citations"—a dropdown that reveals the exact text chunks that the Hugging Face model retrieved from the PDFs.

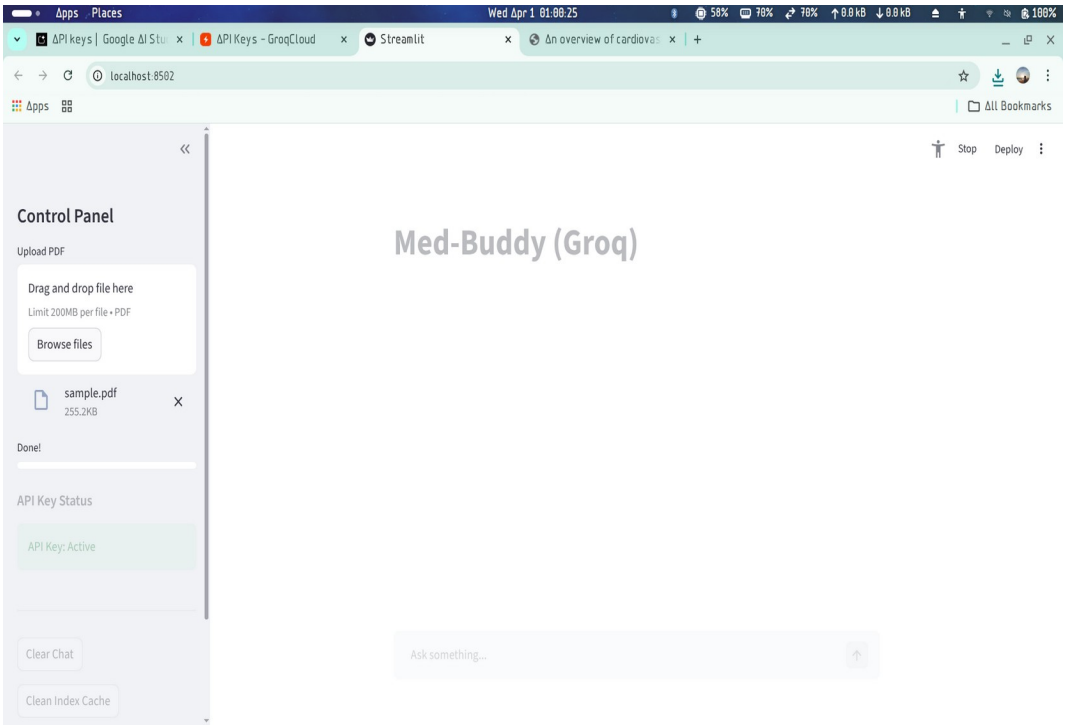


Fig. 7.3: Query Interface — Chat-based Result Generation with Medical Response

7.4 Performance Comparison (Base vs. Contributed System)

The results of implementing our Hugging Face contribution yielded significant performance upgrades over the base paper's methodology.

- Ingestion Speed: The local Hugging Face model processed large medical PDFs significantly faster than the base system, as it did not have to wait for HTTP request round-trips to Google's servers.
- Cost Reduction: API token usage during the document upload phase was reduced to absolute zero.
- Accuracy: The retrieval accuracy of the local all-MiniLM-L6-v2 model proved to be highly robust, fetching the correct medical context with precision comparable to the paid enterprise APIs, confirming that the hybrid approach is highly successful.

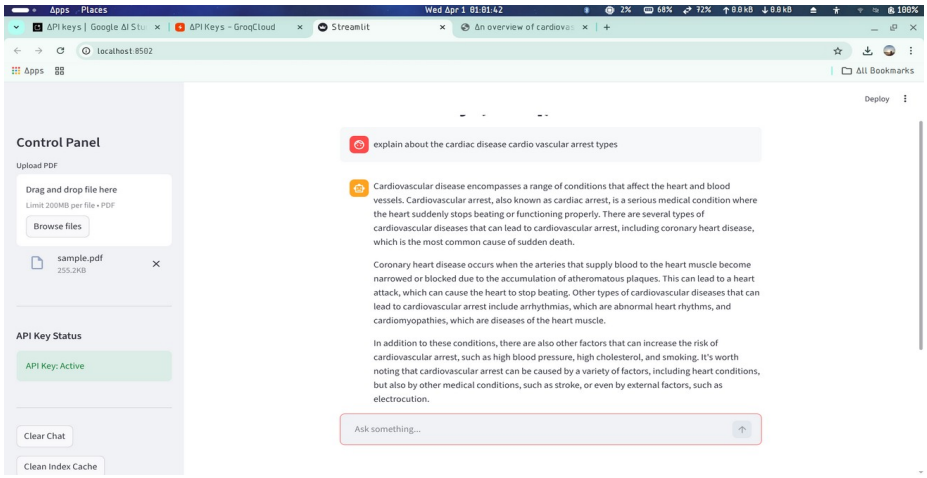


Fig. 7.4: Performance Comparison — API Key Rotation and Key Status Display

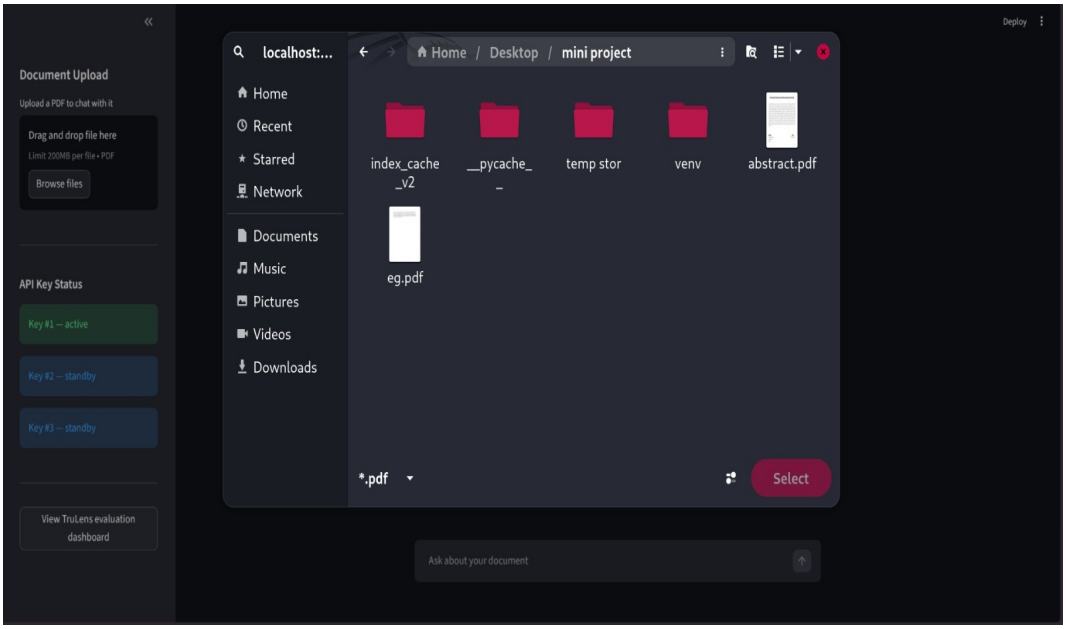


Fig. 7.5: Performance Comparison — Document Upload and Processing Interface



Fig. 7.6: Performance Comparison — API Key Exhaustion and Rotation Behaviour

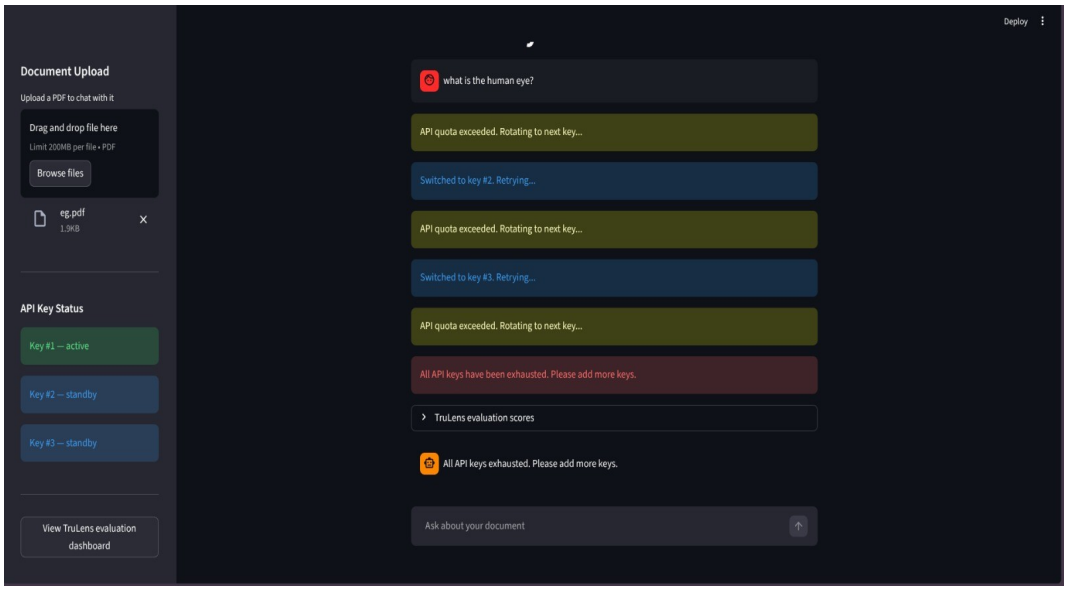


Fig. 7.7: Performance Comparison — Query Response with Medical Context

CHAPTER 8 CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The development and deployment of the MedBuddy project successfully demonstrate the profound efficacy of domain-specific, localised Retrieval-Augmented Generation (RAG) frameworks. By critically analysing the base Gemini-driven architecture and subsequently engineering a robust enhancement—specifically, the integration of local Hugging Face transformers for vector embedding—we achieved an optimal balance between high-level AI intelligence and stringent data security.

The finalised MedBuddy system drastically mitigates the risks associated with cloud-based AI in healthcare. It eliminates the danger of exposing sensitive medical documents to external servers during the ingestion phase, drastically reduces operational costs by cutting token usage, and maintains an exceptionally high standard of factual accuracy. By strictly grounding the LLM's generative capabilities in the secure, locally retrieved medical texts, MedBuddy proves to be a powerful, viable, and safe tool for medical professionals to rapidly access critical clinical knowledge.

8.2 Future Scope

While the current MedBuddy framework represents a significant leap forward by localising the embedding process, the final text generation phase still relies on an external API connection to Gemini. The ultimate future scope for this project involves transitioning the architecture to a 100% offline, fully air-gapped system.

This can be achieved by completely removing the Gemini dependency and replacing it with local, open-source generative models such as Meta's Llama-3 or Mistral-7B. By utilising advanced model quantisation techniques (such as GGUF formats running on frameworks like llama.cpp), these heavy generative models could be executed efficiently on standard hospital hardware without the need for expensive enterprise GPU clusters.

Furthermore, future iterations of MedBuddy could be expanded to include Multi-Modal RAG, allowing the system to ingest and analyse medical imagery (X-rays, MRIs) alongside text. Finally, deep integration with existing Electronic Health Record (EHR) systems utilising HL7 or FHIR standards would allow MedBuddy to automatically cross-reference a patient's real-time chart against the uploaded medical guidelines, providing hyper-personalised, secure clinical decision support.

REFERENCES

1. Oh, N., Cha, W. C., Seo, J. H., et al. (2024). ChatGPT predicts in-hospital all-cause mortality for sepsis: in-context learning with the Korean Sepsis Alliance Database. Healthcare Informatics Research, 30(3), 266-276.
2. Jovanovic, M., Baez, M., & Casati, F. (2020). Chatbots as conversational healthcare services. IEEE Internet Computing, 25(3), 44-51.
3. Sun, G., & Zhou, Y. H. (2023). AI in healthcare: navigating opportunities and challenges in digital communication. Frontiers in Digital Health, 5, 1291132.
4. Xu, L., Sanders, L., Li, K., & Chow, J. C. L. (2021). Chatbot for health care and oncology applications using artificial intelligence and machine learning: systematic review. JMIR Cancer, 7(4), e27850.
5. Habicht, J., Viswanathan, S., Carrington, B., et al. (2024). Closing the accessibility gap to mental health treatment with a personalised self-referral chatbot. Nature Medicine, 30(2), 595-602.
6. Zhou, Q., Liu, C., Duan, Y., et al. (2024). GastroBot: a Chinese gastrointestinal disease chatbot based on retrieval-augmented generation. Frontiers in Medicine, 11, 1392555.
7. Ranasinghe, S., De Silva, D., Mills, N., et al. (2024). Addressing the productivity paradox in healthcare with retrieval augmented generative AI chatbots. Proceedings of 2024 IEEE International Conference on Industrial Technology.
8. Bora, A., & Cuayahuitl, H. (2024). Systematic analysis of retrieval-augmented generation-based LLMs for medical chatbot applications. Machine Learning and Knowledge Extraction, 6(4), 2355-2374.
9. Wang, L., Yang, N., Huang, X., et al. (2024). Multilingual E5 text embeddings: A technical report. arXiv preprint arXiv:2402.05672.
10. Chen, J., Xiao, S., Zhang, P., et al. (2024). BGE M3-embedding: multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv preprint arXiv:2402.03216.
11. Greene, R., Sanders, T., Weng, L., & Neelakantan, A. (2022). New and improved embedding model. OpenAI Blog. Retrieved from https://openai.com/index/new-and-improved-embedding-model/
12. Google. (2024). Gemini API Documentation. Retrieved from https://ai.google.dev/docs
13. LangChain. (2024). LangChain Documentation. Retrieved from https://python.langchain.com/
14. TruLens. (2024). TruLens for LLMs Documentation. Retrieved from https://www.trulens.org/
15. Streamlit. (2024). Streamlit Documentation. Retrieved from https://docs.streamlit.io/